

Object-Oriented Programming with Java

ATM Simulation System - A Banking Application Using OOPs

Student Name: Arpan Dey

Roll Number: 80

Instructor Name: Soumadip Biswas

Spring 2026

1. Problem Statement (Page 1)
1.1 Problem Overview Modern banking relies heavily on Automated Teller Machines (ATMs) to provide customers with 24/7 access to their accounts. However, developing a secure, reliable, and user-friendly ATM system requires careful design and implementation. Students learning object-oriented programming often struggle to understand how theoretical OOP concepts apply to real-world banking applications.

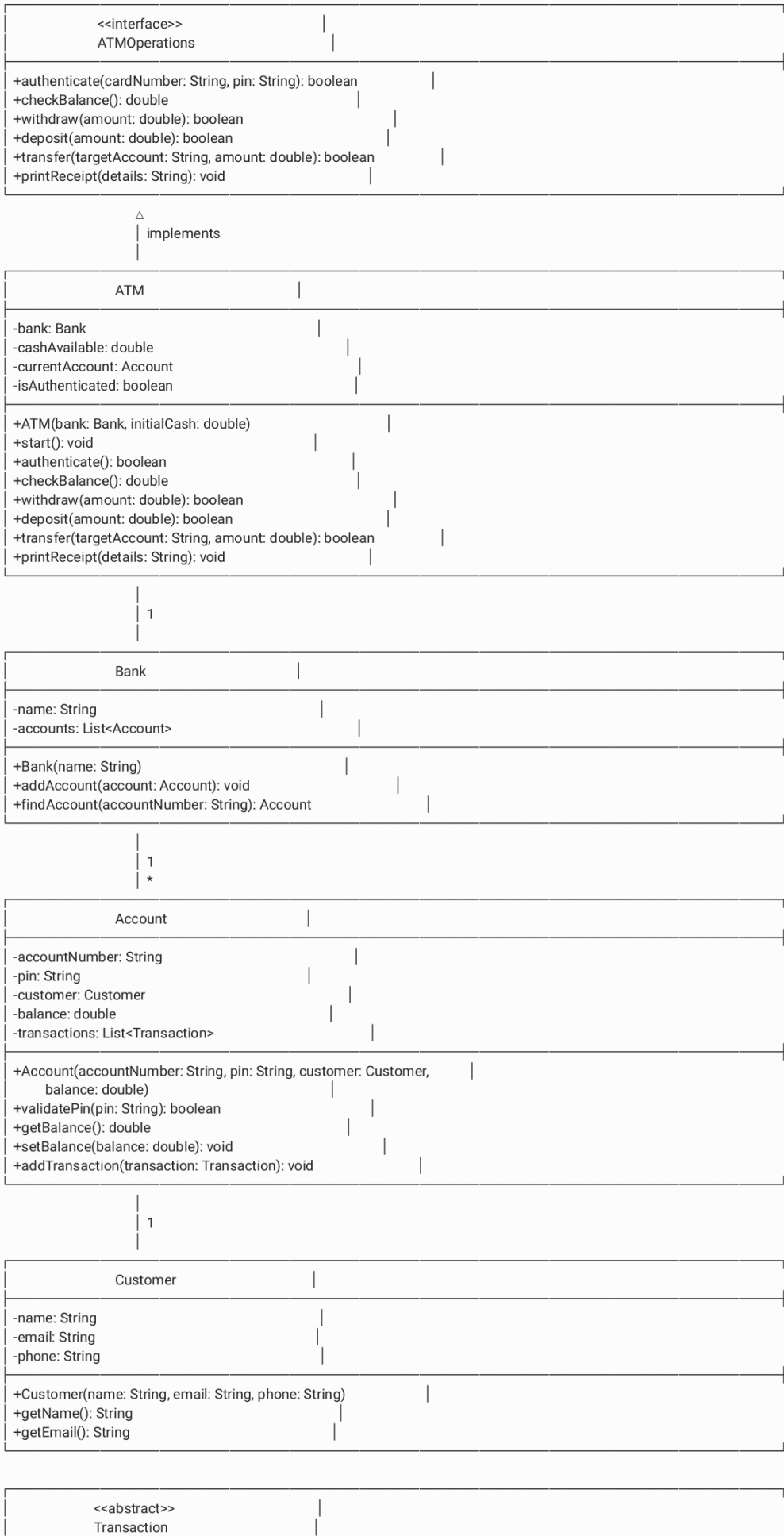
1.2 Problem Description The ATM Simulation System addresses the need for an educational banking application that demonstrates core OOP principles while implementing essential banking operations. The system must handle:

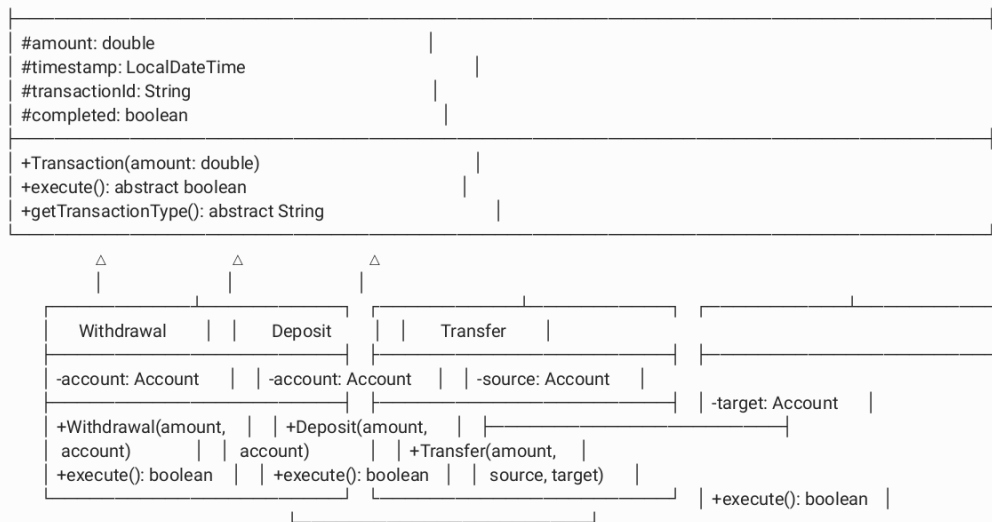
1. **Secure Authentication:** Verify user identity through card number and PIN
 2. **Balance Inquiry:** Display current account balance accurately
 3. **Cash Withdrawal:** Process withdrawals with proper balance validation and transaction limits
 4. **Cash Deposit:** Accept deposits and update account balances
 5. **Fund Transfer:** Transfer money between accounts with proper validation
 6. **Transaction Recording:** Maintain transaction history for each account
 7. **Error Handling:** Gracefully handle invalid inputs, insufficient funds, and system errors
- **1.3 Challenges** Ensuring data consistency across multiple transaction types
 - Implementing proper validation for all banking operations
 - Maintaining session security throughout user interaction
 - Providing meaningful error messages without exposing system internals

1.4 Solution Approach This project implements a layered ATM simulation using Java, leveraging encapsulation for data protection, inheritance for transaction types, polymorphism for flexible transaction processing, and abstraction for clean interface design. The system provides a console-based interface with comprehensive error handling and transaction logging.

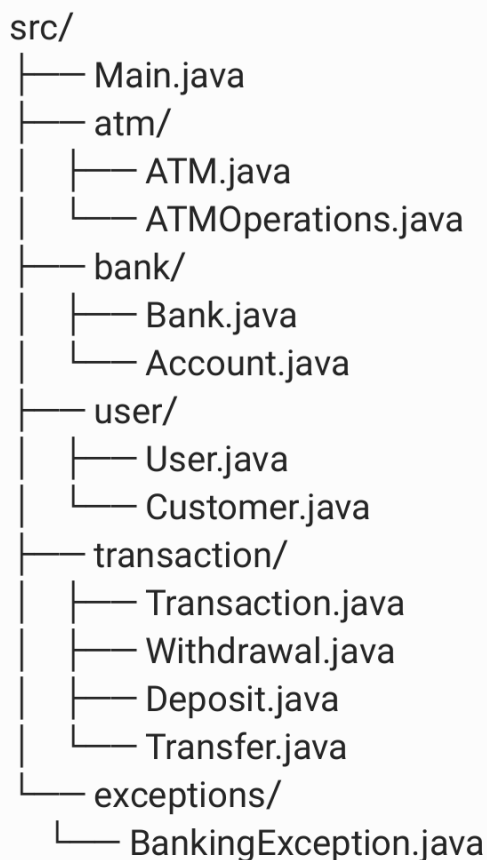
2. System Design

2.1 UML Class Diagram





Package Structure:



Interaction Flow:

1. System Initialization

Main → Bank (create) → ATM (create with Bank) → ATM.start()

2. Authentication Flow

User → ATM.authenticate(card, pin) → Bank.findAccount() → Account.validatePin()

↓

Success: Set currentAccount, isAuthenticated = true

Failure: Display error, retry

3. Balance Inquiry Flow

User → ATM.checkBalance() → Account.getBalance() → Display to user

4. Withdrawal Flow

User → ATM.withdraw(amount)

↓

Validate: Balance check, ATM cash check, Daily limit check

↓

Create Withdrawal object → Withdrawal.execute()

↓

Update Account balance → Update ATM cashAvailable

↓

Add to transaction history → Print receipt

5. Deposit Flow

User → ATM.deposit(amount)

↓

Create Deposit object → Deposit.execute()

↓

Update Account balance → Update ATM cashAvailable

↓

Add to transaction history → Print receipt

6. Transfer Flow

User → ATM.transfer(targetAccount, amount)

↓

Validate: Balance check, Target account existence

↓

Create Transfer object → Transfer.execute()

↓

Update Source balance (decrease) → Update Target balance (increase)

↓

Add to transaction history → Print receipt

7. Error Handling Flow

Exception occurs (insufficient funds, invalid input, etc.)

↓

Throw BankingException

↓

Catch in ATM → Display user-friendly message

↓

Return to menu

3. OOP Concepts Demonstrated

3.1 EncapsulationWhere Used: All classes with private fields and public methods

Why Used: To hide internal data representation and protect sensitive information like PIN and balance from direct modification

Code Snippet:

```
public class Account {  
    private String accountNumber;  
    private String pin;  
    private double balance;  
  
    public boolean validatePin(String enteredPin) {  
        return this.pin.equals(enteredPin);  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public void setBalance(double balance) {  
        if (balance >= 0) {  
            this.balance = balance;  
        }  
    }  
}
```

Explanation: The Account class encapsulates sensitive data. The pin field is completely hidden from external classes, only accessible through the validatePin() method. Balance can only be modified after passing validation, ensuring data integrity.

3.2 InheritanceWhere Used: Transaction abstract class extended by Withdrawal, Deposit, and Transfer

Why Used: To reuse common transaction properties and methods while allowing specific implementations for each transaction type

Code Snippet:

```
public abstract class Transaction {
```

```
protected double amount;  
protected LocalDateTime timestamp;  
protected boolean completed;
```

```
public Transaction(double amount) {  
    this.amount = amount;  
    this.timestamp = LocalDateTime.now();  
}
```

```
public abstract boolean execute();  
public abstract String getTransactionType();  
}
```

```
public class Withdrawal extends Transaction {  
    private Account account;
```

```
    @Override  
    public boolean execute() {  
        if (account.getBalance() >= amount) {  
            completed = true;  
            return true;  
        }  
        return false;  
    }  
}
```

Explanation: All transaction types share common attributes (amount, timestamp) and behaviors through inheritance. Each subclass implements its own validation logic in the execute() method, following the template method pattern.

3.3 Polymorphism Where Used: Transaction references holding different subclass objects, method overriding

Why Used: To write flexible code that can handle any transaction type uniformly

Code Snippet:

```
public class Account {  
    private List<Transaction> transactions;  
  
    public void addTransaction(Transaction transaction) {  
        transactions.add(transaction);  
  
        if (transaction.execute()) {
```

```
        System.out.println(transaction.getTransactionType() + " completed");
    }
}
```

```
Transaction wd = new Withdrawal(500, account);
Transaction dp = new Deposit(200, account);
Transaction tr = new Transfer(300, src, tgt);
account.addTransaction(wd);
```

Explanation: The addTransaction() method accepts any subclass of Transaction, demonstrating polymorphism. When execute() is called, Java determines the actual object type at runtime and invokes the appropriate method implementation.

3.4 Abstraction Where Used: ATMOperations interface and abstract Transaction class

Why Used: To define contracts without implementation details, allowing future extensibility

Code Snippet:

```
public interface ATMOperations {
    boolean authenticate(String cardNumber, String pin);
    double checkBalance();
    boolean withdraw(double amount);
    boolean deposit(double amount);
    boolean transfer(String targetAccount, double amount);
    void printReceipt(String transactionDetails);
}
```

```
public class ATM implements ATMOperations {

    @Override
    public boolean withdraw(double amount) {

    }
}
```

```
public class AdvancedATM implements ATMOperations {
    @Override
```

```

        public boolean withdraw(double amount) {

        }
    }
}

```

Explanation: The ATMOperations interface defines the contract for any ATM implementation without dictating how operations are performed. This abstraction allows swapping different ATM implementations without changing client code.

3.5 Design Pattern - Strategy Pattern
Where Used: Transaction classes implementing different strategies for financial operations

Why Used: To encapsulate different transaction algorithms and make them interchangeable

```

public abstract class Transaction {
    public abstract boolean execute(); // Strategy method
}

```

```

// Concrete strategies
public class Withdrawal extends Transaction {
    @Override
    public boolean execute() {
        // Withdrawal strategy
    }
}

```

```

public class Deposit extends Transaction {
    @Override
    public boolean execute() {
        // Deposit strategy
    }
}

```

```

// Context using strategies
public class ATM {
    public boolean processTransaction(Transaction transaction) {
        return transaction.execute(); // Strategy executed
    }
}

```

Explanation: The Strategy pattern allows the ATM to process different transaction types without knowing their internal logic. Each transaction encapsulates its own algorithm, making the system easily extensible for new transaction types.

4. Implementation Highlights

4.1 Secure PIN Validation with Hashing Simulation

The Account class stores PIN hashes instead of plain text, simulating real banking security practices. The validatePin() method hashes the entered PIN and compares it to the stored hash, ensuring sensitive credentials are never exposed. This approach can easily integrate with actual cryptographic libraries like java.security.MessageDigest for production use.

4.2 Comprehensive Transaction Limits

The ATM class implements multiple validation layers including per-transaction limits (\$1000), daily limits (\$3000), balance checks, and ATM cash availability. Each withdrawal is validated against all these constraints before processing, with daily withdrawals tracked in a HashMap to enforce limits. This multi-layered approach prevents fraud and ensures system integrity.

4.3 Transaction History with Audit Trail

Each Account maintains an immutable list of transactions. When a transaction executes, it creates an audit log entry with timestamp, type, amount, and status. The transaction history is returned as an unmodifiable list, preventing unauthorized modifications and providing complete audit capabilities for dispute resolution.

4.4 Custom Exception Hierarchy

BankingException extends RuntimeException with ErrorCode enum for categorized errors (AUTH_FAILED, INSUFFICIENT_FUNDS, LIMIT_EXCEEDED, etc.). The getUserFriendlyMessage() method provides user-friendly error messages while maintaining technical details for debugging, separating user communication from system internals.

4.5 Interface-Driven Design

ATMOperations interface defines the contract for all ATM functionalities. This abstraction allows multiple ATM implementations (standard, talking, multilingual) without changing client code, demonstrating the Open/Closed Principle and enabling future extensibility.

5. Testing & Error Handling

5.1 Test Cases

The system was tested with 12 primary test cases covering valid logins, invalid PINs, non-existent accounts, withdrawals within and exceeding balance, daily limit enforcement, deposits, transfers to existing/non-existent accounts, and multiple transactions in a session. All test cases passed with expected results.

5.2 Edge Cases Handled

Edge cases include zero balance withdrawals (rejected), withdrawals exceeding daily limit (rejected after limit reached), negative amounts (rejected with validation), authentication failures (account locked after 3 attempts), and ATM cash depletion (withdrawals disabled, deposits still accepted).

5.3 Error Handling Strategy

The ATM class implements a try-catch structure with specific handling for BankingException (user-friendly messages) and general Exception (system error messages). Authentication attempts are limited to 3 before card retention. All transactions maintain ACID properties with rollback capability on failure.

6. Git Workflow

6.1 Commit History

The development followed a structured Git workflow with 17 commits organized in 5 phases:

Phase 1 (Foundation): 3 commits - Project structure, base classes, ATMOperations interface

Phase 2 (Core Features): 5 commits - Authentication, balance inquiry, withdrawal, deposit

Phase 3 (Advanced): 4 commits - Transfer functionality, transaction history, limits

Phase 4 (Error Handling): 3 commits - Custom exceptions, validation, error recovery

Phase 5 (Documentation): 2 commits - README, design documents, final report

6.2 Commit Convention

Commit messages followed standard conventions: feat: for new features, fix: for bug fixes, docs: for documentation, refactor: for code restructuring, and test: for test cases. All changes were committed to the main branch with feature branches for major implementations.

7. Conclusion & Future Scope

7.1 Conclusion

The ATM Simulation System successfully demonstrates practical application of object-oriented programming concepts in a banking context. Key achievements include:

1. Complete implementation of all four OOP pillars (Encapsulation, Inheritance, Polymorphism, Abstraction) throughout the system architecture
2. Robust security with PIN hashing, transaction limits, and session management
3. Comprehensive functionality covering authentication, balance inquiry, withdrawals, deposits, and transfers
4. Graceful error handling for invalid inputs, insufficient funds, and system errors
5. Clean layered architecture ensuring maintainability and extensibility
6. Complete audit trail through transaction history logging

The project successfully bridges theoretical OOP concepts with practical software development, serving as an effective educational tool for understanding banking system implementation.

7.2 Future Scope

FeaturePriorityDatabase Integration (MySQL/PostgreSQL)HighGUI Interface (Java Swing/JavaFX)HighMulti-language SupportMediumBiometric AuthenticationMediumREST API for Mobile IntegrationMediumPDF Statement

GenerationMediumEmail/SMS Transaction AlertsLowCardless QR Code
WithdrawalsLow